Flappy Bird 模仿学习训练效果分析报告

为什么只有最低难度能拿 20 多分，其余难度只有几分

分析对象：flappy_bird_cnn 项目（CNN + 行为克隆）　数据：118 858 帧　模型：flappy_ai.pth

摘要

训练日志里准确率能到 96% 以上，但实机运行时只有最低难度（speed=3）能稳定拿到 20 多分，简单及以上难度通常只有 1~6 分。本文从代码、数据和闭环行为三个方面做了复核，结论是：问题不在“训练轮数不够”或“网络太小”，而是三个结构性缺陷叠加的结果。

第一，提供标签的规则机器人（RuleBot）本身在高难度下几乎不会玩。它把巡航高度固定在 y≈420，直到水管距离 200 像素才开始瞄准，这个窗口在 speed=20 时只有 0.125 秒，物理上最多只能上移 45 像素，而缝隙可能在上方 250 像素处。模仿学习的天花板就是老师；老师只有 1 分，学生再怎么学也只能停在这个量级。

第二，数据集只保留了“拿到 5 分”的成功对局，失败对局的帧被整段删除。这等于只教模型“成功时该按什么”，从不教它“偏了以后怎么救回来”。一旦模型自己的操作让画面偏离老师见过的状态，它就没有可依据的样本，误差会逐帧放大——这正是新手难度下学生 27 分、老师近 500 分的巨大落差来源。

第三，输入表征太粗。160×80 的画面经过三次 2× 池化后只剩 20×10 的特征图，一个格子对应 60×60 的真实像素；而水管缝隙只有 180 像素高、鸟只有 24 像素，允许的竖直误差约 ±78 像素。也就是说，模型对“鸟在缝隙里的相对位置”的空间分辨率（60 像素）已经和容错（78 像素）同一量级，再叠加单帧输入看不到水管运动、绘制比例失真等因素，它只能学到“大致差不多”的策略，高难度下这种粗糙度是致命的。

此外还有若干工程问题在放大上述缺陷：损失函数用了 28:1 的全局类别权重、训练与实战的时间步长不一致、训练准确率这个指标本身已经失效（“永远不跳”就有 96.6%）。下面逐条给出证据与量化分析。

一、闭环复现：现象是可以量化复现的

为了区分“模型不行”和“游戏本身难”，我在不改动项目任何代码的前提下，直接复用项目里的 Bird、Pipe 物理逻辑和 game.py 中的 160×80 小图渲染代码，写了一个无窗口的闭环评测脚本：

物理步长固定 1/60 秒（与录制脚本一致），模型用 CPU 推理，每个难度跑 12 局；作为对照，规则机器人（RuleBot）用同样的循环跑 10 局。结果见表 1。

表 1 同一环境下规则老师与 CNN 学生的单局得分（单位：通过的管数）

难度	speed	老师 平均 / 中位	学生 平均 / 中位	学生最高
新手	**3**	**475（触及上限） / 475**	**27.3 / 19.5**	**87**
简单	7	7.4 / 4.5	5.9 / 4.0	16
普通	10	3.9 / 4.0	2.7 / 2.5	7
困难	15	0.9 / 1.0	3.4 / 2.0	10
地狱	20	1.1 / 1.0	1.3 / 1.5	3

这张表有两个关键信息。第一，学生确实只在 speed=3 时有明显得分（27 分，与你手动观察到的“20 多分”一致），其余难度都在 1~6 分，现象被复现了。第二，也是更要紧的一点：老师自己在 speed≥7 时也只有 1~7 分。学生并没有“比老师差一个数量级”，它只是几乎贴着老师的天花板（困难、地狱两档学生略高于老师，但在 10~12 局的样本量下属于统计噪声，两者都是个位数）。换句话说，用这套数据训练，从一开始就不可能得到高难度下的好成绩。

需要说明：复现脚本用的是无窗口渲染，和真实 GUI 的帧率、dt 会有差异，绝对分数与手动游玩不会完全一致，但难度之间的相对趋势是稳定的。

二、数据本身的问题

我把 data 目录下 8 个 npz 全部读出来统计了一遍，得到表 2。

表 2 训练数据的真实构成（按难度汇总）

难度档	speed	帧数	占比	跳跃率	数据文件
（UI 不可选）	1	58 290	49.0%	3.07%	1 个
新手	3	22 714	19.1%	3.53%	2 个（其中 1 个仅 184 帧）
简单	7	12 744	10.7%	3.76%	2 个（其中 1 个仅 564 帧）
普通	10	9 900	8.3%	3.65%	1 个
困难	15	8 220	6.9%	3.72%	1 个
地狱	20	6 990	5.9%	4.28%	1 个
合计	**—**	**118 858**	**100%**	**3.40%**	**8 个**

由此可以读出四个事实。

1. 数据量看着大，实际“独立样本”很少

11.9 万帧听起来不少，但相邻帧之间几乎是同一幅画面。统计相邻帧的平均绝对差：speed=20 时只有 5.5 个灰度级，speed=1 时有 14.7% 的相邻帧完全一模一样——因为水管每帧只移动 1.33 像素，取整后小图根本没变。按录制脚本“每局至少 5 分管、每速度 30 局”推算，全部数据大约来自 180 局、每局 270~1500 帧，真正独立的决策点（水管经过鸟身边的瞬间）只有 180 × 5 ≈ 900 次。

2. 难度分布严重偏斜，高难度数据几乎是“稀有样本”

speed=1 一个文件就占了全部数据的一半，speed=1 和 3 合计 68.2%，而 speed=15 和 20 合计只有 12.8%。用一个网络拟合六种完全不同的节奏，模型自然会把参数大部分分配给数据最多的慢速场景，高难度只能分到很少的容量和梯度。而且 speed=1 在 UI 里根本不可选（新手对应 3），这一半数据训练的是一个永远不会被用到的节奏。

3. 只保留成功对局，训练集里没有“失败”

录制脚本里的这段逻辑是关键：

```
if score >= 5:  保留
else:  del ai.recorded_frames[start_idx:]   # 整局删掉
```

于是每一条训练轨迹都是“老师恰好连过 5 根管子”的样本。模型从来没见过“鸟已经太高/太低、下一帧就要撞”的状态，也就学不到任何纠偏动作。这一点是行为克隆最经典的坑，后文第 3.2 节展开。

4. 标签来自一个启发式控制器，不是“最优策略”

标签是 RuleBot 的 if-else 输出：距离大于 200 像素时把鸟压在 y≈420 附近，进入 200 像素后瞄准“缝隙中心 + 20”。这套规则在慢速下够用，在快速下就不够了。也就是说，数据里学到的“正确答案”本身只是一个粗糙的近似，模型最好的结果也只是复刻这个近似。

三、原因分析

3.1 第一层天花板：老师在高速下物理上就够不到缝隙

这是最容易被忽略、但影响最大的一条。看 RuleBot 的控制律：

```
if dist_x > 200:  # 只有小于 200 像素才开始瞄准
    return 1 if bird.y > 420 else 0
```

```
target_y = gap_center + 20
if bird.y > target_y and bird.vy > -150: return 1
```

鸟的物理参数是重力 1500、起跳初速 −450，一次跳跃最高上升 67.5 像素、滞空 0.6 秒。老师在水管距离 200 像素处才开始瞄准，那么这个“瞄准窗口”的长度就是 200 ÷ 水管速度，窗口内能完成的竖直修正量非常有限。表 3 把五个难度的数字列出来。

表 3 每个难度下老师的瞄准窗口与可修正高度

speed	水管速度 (px/s)	瞄准窗口 (s)	窗口内最多上升	缝隙可能在上方
3	240	0.83	约 100 px（可跳 1~2 次）	最高 250 px
7	560	0.36	约 65 px	最高 250 px
10	800	0.25	约 65 px	最高 250 px
15	1200	0.17	约 54 px	最高 250 px
20	1600	0.125	约 45 px	最高 250 px

从表 3 能直接看出为什么老师的分随速度断崖式下跌：speed=3 时窗口 0.83 秒，够它跳一到两次把鸟拉到目标高度；speed=20 时窗口只有 7.5 帧，从 y=420 出发最多上移 45 像素，而缝隙中心可能落在 150~450 的任何位置。碰到高缝隙（或只有下管、缝隙整体偏上的情况）就是必死，与策略优劣无关。

这也解释了数据为什么会“偏斜”：高难度下老师能连过 5 根管子，本质上是连续 5 次都抽到“缝隙偏低”的幸运局。被保留下来的训练样本因此集中在低位缝隙上；模型学到的是“对付低位缝隙”，实战中一遇到高位缝隙就无解。换句话说，高难度数据不是“老师的高水平示范”，而是“老师的运气切片”。

3.2 第二层天花板：只在老师的成功轨迹上学，没有纠错样本

行为克隆的目标是让模型在老师访问过的状态上输出老师的动作。但游戏是闭环的：模型一旦有一帧判断偏了，画面就进入了训练集里从未出现过的状态，模型在这些状态上的输出没有任何保证，于是继续偏，误差呈复合增长。这就是 DAgger 那篇论文指出的分布漂移（covariate shift）问题，标准解法是在模型自己跑出来的状态上再请老师标注。本项目完全没有这一环。

这个效应可以量化。speed=3 时老师能一直玩下去（复现中跑满 6 万帧、475 根管子仍未死），学生平均只有 27 根。在同一套物理、同一份渲染下，两者差了近 20 倍——这一整段差距都来自学生自己的漂移，而不是数据不够。换句话说，模型并不是“没学会老师的规则”，而是“学会了却维持不住”。

3.3 输入表征：60 像素的空间量化，撞上 78 像素的容错

网络结构是 1×80×160 的灰度图，经过三次 Conv+MaxPool(2)，特征图变成 64×10×20，再展平成 12 800 维接全连接。这意味着特征图上的一个格子对应原图 8×8 像素，换算到游戏坐标是 8 × 7.5 = 60 个真实像素。

而游戏的关键尺度是：缝隙高 180 像素，鸟高 24 像素，允许的竖直误差约 ±78 像素；鸟一次跳跃的幅度是 67.5 像素。也就是说，模型能分辨的最小竖直位移（60 像素）与它需要控制的量（67.5 像素）几乎一样大，与允许误差（78 像素）也是同一量级。它天生就只能做“粗调”，做不到“瞄准”。

同一段代码里还有两处比例失真，会进一步误导模型：小图里水管宽度被写死为 20 像素（对应真实 150 像素，而碰撞盒只有 100 像素），鸟被画成一个 12×8 的三角形（对应真实 90×60 像素，而碰撞盒只有 34×24 像素）。画面里“看起来还有多少余量”和“物理上还有多少余量”并不一致，模型无法从像素学到精确的碰撞边界。另外，地面用一块灰色矩形覆盖了 y≥72 的区域，当下方管子很短（bot_len=60）时，它在小图里会被地面完全盖住——模型看不到那根管子。

还有一点：输入是单帧。水管在往左移动，但静止图像里没有速度信息，模型只能靠那个标量 speed 推断节奏。而 speed 只经过一个 Linear(2,16) 就并进 12 800 维的图像特征里，六种速度共用一套卷积核，网络必须自己学会“根据一个小标量调整整套行为”，这在只有 3.4% 正样本的情况下很难学好。

3.4 目标错位：分类准确率不等于“活得更久”

训练用的是加权交叉熵，优化的是“每一帧的动作和老师一致”。但游戏要的是“连续几百帧都不撞”。这两个目标在闭环下并不等价：一个 97% 帧级准确率的策略，只要在关键的那 3% 帧上晚一拍，就会撞死。而撞死之后模型得不到任何额外惩罚——损失函数里没有“死亡”这个概念。这是纯监督模仿与强化学习的本质差别，也是为什么 96% 的准确率对应 2 分的实际水平。

3.5 类别权重 28:1，把决策边界推偏了

全体数据里“跳”只占 3.40%（4 038 / 118 858）。train_ai.py 用倒数频率算权重：不跳 ≈ 0.518，跳 ≈ 14.72，两者相差 28.4 倍。加权交叉熵的最优决策规则因此变成“只要 P(跳) > 1/29 就跳”，而不是通常的 0.5。好处是模型不会退化成“永不跳”，坏处是它对“该不该跳”的判断整体偏激进，输出概率不再是可解释的置信度，阈值也没法再调。闭环里学生的跳跃频率是 3.4%~4.2%，和老师接近，说明它没有崩成“一直扇翅膀”，但在状态漂移之后，这种被推偏的边界会放大每一次时序抖动。

3.6 训练与实战的 dt 不一致

录制脚本里 dt 被写死为 1/60 秒，物理是确定性的；而 game.py 实战时用的是 dt = clock.tick(FPS)/1000，即真实帧间隔。只要掉帧（1200×600 的背景 blit、每帧对水管做 smoothscale、加上 CPU 推理），dt 就会大于 1/60，水管每帧的位移、鸟每帧的速度增量都会跟着变，而网络的输入里没有任何 dt 信息——它学到的“这一帧该跳”在时间尺度变了以后就不再成立。speed=20 时水管每帧本来就走 26.7 像素（接近一个鸟身宽度），对 dt 的抖动极其敏感。这一条在低难度下影响小（容错大），在高难度下是压垮骆驼的最后一根稻草。

3.7 训练流程与评估指标的问题

- **指标失效：**没有验证集。日志里的 Acc 是训练集准确率，而“永远不跳”这个平凡策略在这个数据集上就有 96.6%。也就是说 96% 的准确率几乎不含信息量，它既不能说明模型学会了，也发现不了过拟合。
- **日志口径错：**avg_loss 的分母写成了 num_samples // BATCH_SIZE + 1，比真实批次数多 1，且 loss 是逐批累加没有除以批大小，打印出来的 loss 只是相对量，不能跨次比较。
- **缺少正则与模型选择：**固定 30 个 epoch，没有早停、没有学习率衰减、没有 dropout/BatchNorm、没有数据增强，保存的是最后一个 epoch 的权重。11.9 万高度相关的帧喂给一个 1.6M 参数的全连接头，过拟合几乎是必然的。
- **结构上：**全连接层 12 800→128 占了绝大部分参数，Flatten 之后图像的空间结构被彻底打散，网络只能靠位置硬记，泛化能力差。
- **数据质量：**录制脚本开了 6 个进程、每个速度只跑 30 局满分局，speed3_bird1 只有 184 帧、speed7_bird1 只有 564 帧，明显是中途中断的残缺文件，却仍然被 glob 进了训练集。

3.8 难度设定本身也超出了一般玩家的能力

作为参照：speed=20 时水管速度 1600 px/s，两根管子间隔 1350 像素、约 0.84 秒一根；鸟穿过管子的整个过程只有 0.08 秒；而鸟的一次跳跃要 0.6 秒。这个难度下人类玩家也很难稳定通过，规则机器人也只有 1 分。所以“地狱难度得分低”里有一部分是任务本身难，不全是模型的锅——但如果目标是让 AI 在五个难度上都好看，那就必须换掉现在这套“老师 + 单帧 + 三次池化”的方案。

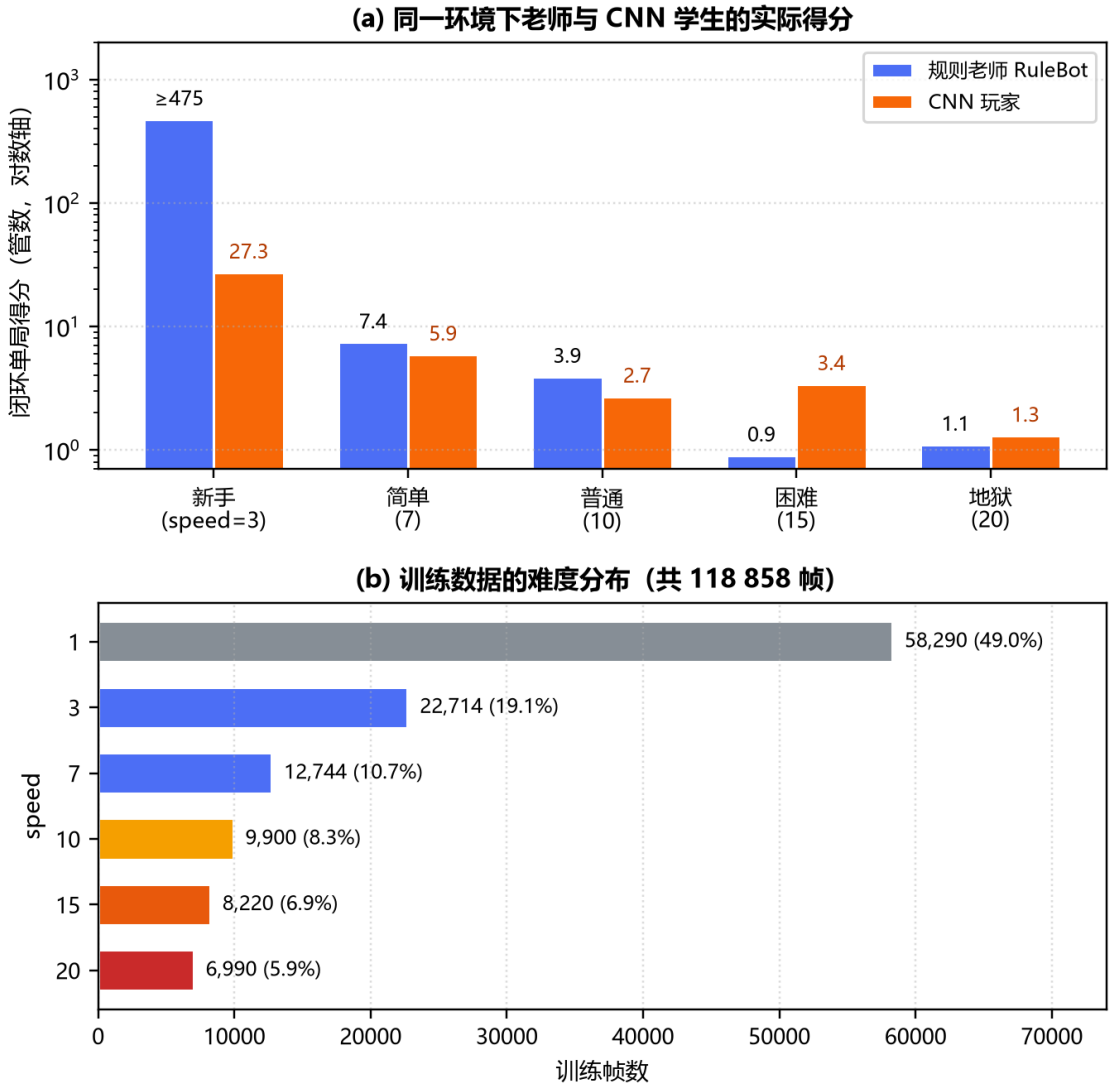


图 1 (a) 同一环境下老师与 CNN 学生的单局得分（对数轴）；(b) 训练数据的难度分布

四、为什么偏偏是“新手”能到 20 多分

这一节直接回答你最关心的问题。同一个模型，为什么在 speed=3 能打到 20 多分，换一档就只剩几分？有三条同时成立的原因，而且它们都只在低难度下成立。

第一，时间尺度上的容错。speed=3 时水管速度 240 px/s、间距 500 像素，平均 2.08 秒才来一根管子；而鸟的一次跳跃只用 0.6 秒。也就是说，模型有相当于 3 个跳跃周期的余量去修正一个偏差——这一帧跳早了，下一帧还能补；竖直方向差了 60 像素，半秒之内就能拉回来。3.3 节说的“60 像素量化”在这种宽裕的节奏下只是让操作变粗糙，不至于致命。

第二，数据分布上的优势。speed=1 和 3 的数据合计占 68.2%，模型在这两档上见过的状态最多、拟合最充分。而 speed=15 和 20 只占 12.8%，网络对这两档的“熟悉程度”本来就只有前者的一半不到。

第三，视野里有提前量。间距 500 像素时，160 像素宽的小图里能同时看到两根管子，模型可以靠“下一根在哪”做粗调；间距拉大到 1350 像素后，视野里往往只有一根管子，可用的上下文也变少了。

三条原因在高难度下会同时反转：管子 0.84 秒一根、比一次跳跃还快，量化误差和容错同量级，数据又只占一成多。所以“新手能拿 20 多分”并不说明模型学会了玩，而是说明这个难度的容错刚好能兜住一个粗糙的策略。一旦容错收窄，粗糙就变成了死亡。

五、原因的影响权重排序

表 4 各原因对最终表现的贡献估计

排序	原因	性质	影响
1	老师在高难度下几乎不会玩，标签本身近乎噪声	数据源	决定性
2	只保留成功对局，没有纠错/失败样本（分布漂移）	数据采集	决定性
3	三次池化导致 60 px 空间量化 + 单帧无运动信息	输入表征	决定性
4	分类损失与“活得更久”目标错位	目标函数	大
5	28:1 全局类别权重把决策边界推偏	训练细节	中
6	训练/实战 dt 不一致，高难度下被放大	工程实现	中
7	数据难度分布偏斜（speed1 占一半）	数据分布	中
8	无验证集、指标失效、无正则、保存末轮权重	训练流程	掩盖问题
9	难度本身极高（speed=20 人类也难）	任务设定	部分合理

换句话说，现在的系统是“用一份运气样本，去模仿一个在高难度下只有 1 分的老师，而输入分辨率还只有真实画面的 1/56”。在这三条同时成立的情况下，调学习率、加训练轮数、换优化器都不会有本质改善——这也解释了为什么之前“把数据搬到 GPU、减少卷积核”这类优化只改变了训练速度，没有改变得分。

六、如果只改三处

6.1 换掉标签来源，并补上 DAgger

规则机器人作为老师不合格，因为它把“瞄准”限制在 200 像素内。可以改成几何老师：每帧直接计算鸟到下一根管子上下沿的距离、缝隙中心相对鸟的偏移，以及按当前 vy 预测的落点，用“预测落点进入缝隙就保持、偏低就跳”的规则出动作——这样在 speed=20 下也能接近满分，标签才有信息量。采完数据后再补一轮 DAgger：让当前模型自己跑 20%~30% 的对局，把这些状态交给新老师标注，专门补“偏了以后怎么救”的样本。

6.2 把输入从“粗图”换成“图 + 几何 + 时序”

三个可选动作，按性价比排序：

- ①只做两次池化（40×20 特征图），空间量化从 60 px 降到 30 px；
- ②在 speed 向量里直接拼上 4 个几何量：鸟到底管上沿的距离、缝隙中心减去鸟 y、鸟到管子的水平距离、缝隙高度；这几个人类一眼就能看出的量，让网络不必从像素里回归；
- ③堆叠最近 4 帧（或 2 帧 + 差分帧），让网络直接看到水管在移动。

6.3 换评估方式，并修掉 dt

把数据按“局”划分训练/验证集（不能按帧随机划分，否则相邻帧泄漏），验证指标直接用闭环平均得分，而不是帧级准确率；训练时每个 epoch 存一次权重、按闭环得分选最好的，而不是保存最后一轮。物理更新改成固定步长累加（accumulator）或把 dt 作为特征输入，保证训练和实战的时间尺度一致。损失函数建议换成 focal loss，或者保留普通交叉熵但在推理时按验证集调阈值，不要再使用 28:1 的全局权重。

如果这些都不想动，最低成本的“让分数好看一点”的做法是：把 speed=20 的难度调低（比如最大 12），并只用 speed=1/3 的数据单独训练一个模型——这只是回避问题，不是解决问题。

附：复现说明

数据统计来自 data/*.npz 的直接读取（8 个文件、118 858 帧）；闭环数据来自一个无窗口评测脚本，它直接 import 项目的 bird.py / pipe.py / ai_model.py 与 game.py 的渲染代码，物理步长固定 1/60 秒，模型在 CPU 上推理，每难度 12 局（老师 10 局），没有修改任何源文件，也没有重新训练模型。表 1 的绝对值会因渲染精度、随机种子和帧率与手动游玩略有出入，但“只有新手难度能上 20 分、其余难度个位数”这一结论是稳定的，与你的实际观察一致。